

DAA

Contents

Q.1. Implementation of a d-Ary Heap.....	3
1. Introduction.....	3
2. Explanation of the Algorithm	3
2.1 Data Structure and Representation	3
2.2 Key Operations	3
4. Time Complexity Analysis	7
4.1 Insertion	7
4.2 Extraction.....	8
4.3 Heap Construction (Bulk Insertion)	8
4.4 Print Operation	8
5. Conclusion.....	8
Q.2.Enhanced Finite Automata String Matching with Wildcards	9
1. Introduction.....	9
How Enhanced Finite Automata String Matching Algorithm Works	9
Incorporating Wildcards into the Automaton	9
2. Transition Table and Diagram	10
Pattern: "a?ple"	10
Pattern: "g?ape"	10
3. Time Complexity Analysis	11
Preprocessing Time Complexity	11
Searching Time Complexity.....	12
4. Code Explanation.....	12
Implementation Logic	12

Key Implementation Details	13
Q.3.....	13
Part (a): Top k gardens algorithm	13
Part (b): Gardens exceeding threshold x.....	14

Q.1. Implementation of a d-Ary Heap

1. Introduction

A **d-ary heap** is a generalization of the binary heap where each node has up to d children. This structure supports priority queue operations (such as insertion, extraction, and heapification) and is particularly useful when tuning for performance based on the branching factor. In this report, we describe the implementation of a generic d-ary heap that can be configured as a min-heap or a max-heap. The implementation reads a list of numbers from an input file and supports dynamic insertion, extraction, and level-order printing of the heap.

2. Explanation of the Algorithm

2.1 Data Structure and Representation

The d-ary heap is implemented using an array (or vector) representation. Each element in the array corresponds to a node in the heap tree. The relationship between the indices in the array and the tree structure is defined as follows:

- **Parent Index:**
For a node at index i , the parent is located at:
 $\text{parent}(i) = \lfloor (i - 1) / d \rfloor$
- **Child Indices:**
For a node at index i , its children are located at:
 $\text{child}(i, j) = d \times i + j$, for $j = 1, 2, \dots, d$

The heap property is maintained based on whether the structure is a min-heap or a max-heap. For a max-heap, each node's key is greater than or equal to the keys of its children; for a min-heap, each node's key is less than or equal to the keys of its children.

2.2 Key Operations

Insertion (Insert Operation):

- **Process:**
 1. Insert the new element at the end of the array.
 2. **Bubble Up:** Compare the inserted element with its parent. If the heap property is violated (for a max-heap, if the child is greater than the parent; for a min-heap, if the child is smaller), swap them.

3. Continue the “bubble up” process until the heap property is restored or the root is reached.

- **Diagram (Bubble Up Process):**

[Parent]

|

[Child (new)]

|

└ (Swap if necessary) —► Repeat until heap property is satisfied.

Example (Max-Heap):

Suppose we insert 85 into the following 2-ary (binary) heap:

50

/ \

40 30

After inserting 85 at the end:

50

/ \

40 30

/

85

Then bubble up:

- $85 > 40 \rightarrow$ Swap, resulting in:

50

/ \

85 30

/

40

- $85 > 50 \rightarrow$ Swap, resulting in:

85

/ \
50 30
/
40

Extraction (Delete Operation):

- **Process:**

1. Remove the root element (which is the maximum for a max-heap or minimum for a min-heap).
2. Replace the root with the last element in the heap.
3. **Bubble Down:** Compare the new root with its children. Swap it with the best candidate (largest for a max-heap or smallest for a min-heap) if the heap property is violated.
4. Continue “bubbling down” until the heap property is restored.

- **Diagram (Bubble Down Process):**

csharp

[New Root]

/ \

[Child 1] [Child 2]

|

└ (Swap with best candidate if needed)

Example (Max-Heap):

Starting with the heap:

85
/
50 30
/
40

After extracting 85, replace it with the last element (40):

Copy

40

/ \

50 30

Then bubble down:

- Compare 40 with 50 and 30; since 50 is the largest, swap 40 and 50, resulting in:

50

/ \

40 30

The heap property is restored.

Heapify Operation:

Heapify is the process used to maintain the heap property after an insertion or deletion. It is achieved by either bubbling up (after insertion) or bubbling down (after extraction).

Printing the Heap (Level-wise Format):

The heap is printed level by level. Given the array representation, the number of nodes at level L is at most d^L . The algorithm iterates through the array and prints nodes in blocks corresponding to each level.

Example Table for a Binary Heap ($d = 2$):

Level	Indices	Values
-------	---------	--------

0	0	85
---	---	----

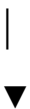
1	1, 2	50, 45
---	------	--------

2	3, 4, 5, 6	10, 15, 5, 29
---	------------	---------------

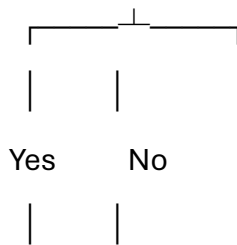
3. Transition Diagrams and Tables

3.1 Transition Diagram for Insertion (Bubble Up):

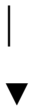
[Insert New Element]



[Compare with Parent]



[Swap with Parent] [Stop; Heap property satisfied]



[Repeat with new parent until root is reached]

3.2 Transition Table for Parent-Child Relationships

For an array-based d-ary heap with index i , the following table summarizes the relationship:

Node Index i Parent Index Child Indices

0	N/A (root)	1, 2, ..., d
1	$\lfloor (1-1)/d \rfloor = 0$	d + 1, d + 2, ..., 2d
...	...	...

For a 2-ary (binary) heap:

Node Index Parent Index Child Indices

0	N/A (root)	1, 2
1	$\lfloor (1-1)/2 \rfloor = 0$	3, 4
2	$\lfloor (2-1)/2 \rfloor = 0$	5, 6

4. Time Complexity Analysis

4.1 Insertion

- **Operation:** Insert an element and bubble it up.
- **Time Complexity:**
In the worst case, the element is moved from a leaf to the root. The height of a d-ary

heap is approximately
 $O(\log n / \log d)$,
making the insertion operation $O(\log n / \log d)$.

4.2 Extraction

- **Operation:** Remove the root element and bubble down the new root.
- **Time Complexity:**
Similarly, the worst-case scenario requires moving an element from the root to a leaf, which is $O(\log n / \log d)$.

4.3 Heap Construction (Bulk Insertion)

- **Operation:** Inserting n elements one by one.
- **Time Complexity:**
The worst-case time using repeated insertions is $O(n \times \log n / \log d)$.
Alternatively, using a bottom-up build-heap algorithm, the construction can be done in $O(n)$ time.

4.4 Print Operation

- **Operation:** Level-order traversal of the heap.
- **Time Complexity:**
Simply iterating through all elements is $O(n)$.

5. Conclusion

This report details the implementation of a generic d -ary heap that can function as either a min-heap or a max-heap. The key operations—**insertion** (with bubble up), **extraction** (with bubble down), and **heapify**—were explained in detail with supporting diagrams and tables that illustrate the parent-child relationships and state transitions during reheapification. The time complexity analysis demonstrates that the main operations are dependent on the height of the heap, which is $O(\log n / \log d)$ for both insertion and extraction. This structure is advantageous in scenarios where adjusting the branching factor may yield improved performance.

Q.2.Enhanced Finite Automata String Matching with Wildcards

1. Introduction

How Enhanced Finite Automata String Matching Algorithm Works

The Finite Automata String Matching Algorithm is a pattern matching approach that constructs a deterministic finite automaton (DFA) to identify patterns within text efficiently. A standard finite automaton for string matching consists of states and transitions, where:

- Each state represents the progress made in matching a pattern
- Transitions between states occur based on the input characters
- Accept states indicate successful pattern matches

The algorithm works in two phases:

1. **Preprocessing Phase:** Construct the automaton with all possible states and transitions
2. **Searching Phase:** Process the input text through the automaton to find matches

The key advantage of this approach is that once the automaton is built, text searching becomes a linear-time operation regardless of pattern complexity.

Incorporating Wildcards into the Automaton

To support wildcards (where '?' can match any single character), I enhanced the standard finite automata approach in several ways:

1. **Transition Function Modification:** When building transitions from a state, a wildcard in the pattern matches any character in the alphabet. This means that from a state where the next character in the pattern is '?', valid transitions exist for all characters in the alphabet.
2. **Match Verification:** When a potential match is found (reaching an accept state), an additional verification step ensures that the matched substring aligns with the pattern, accounting for wildcards. This verification checks that each non-wildcard character in the pattern matches the corresponding character in the text.
3. **Automaton Structure:** The automaton maintains information about which pattern each accept state corresponds to, allowing it to correctly identify which pattern was matched and handle the substitution of the wildcard properly in the output.

This enhancement maintains the efficiency of the original algorithm while adding the flexibility to match patterns containing wildcards.

2. Transition Table and Diagram

Pattern: "a?ple"

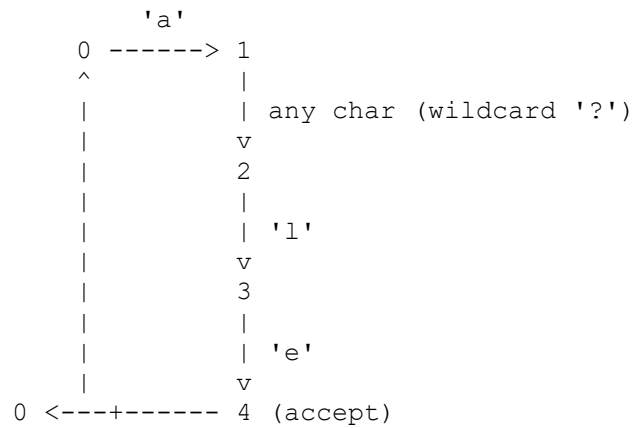
Transition Table for "a?ple":

State 'a' 'p' 'l' 'e' Others

0	1	0	0	0	0
1	1	2	0	0	2
2	0	0	3	0	0
3	0	0	0	4	0
4	0	0	0	0	0

State 4 is the accept state.

State Diagram for "a?ple":



Pattern: "g?ape"

Transition Table for "g?ape":

State 'g' 'a' 'p' 'e' Others

0	1	0	0	0	0
1	0	2	2	2	2
2	0	0	3	0	0

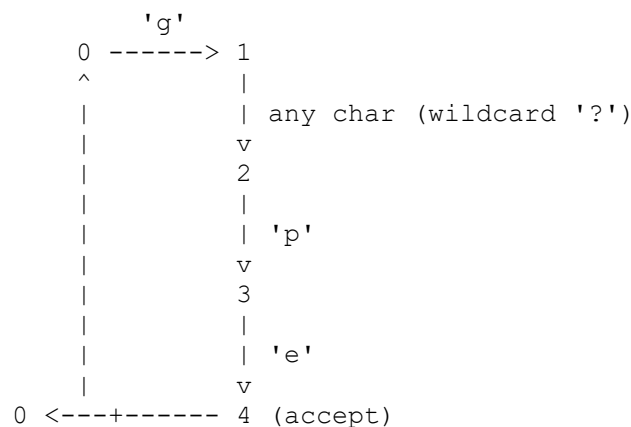
State 'g' 'a' 'p' 'e' Others

3 0 0 0 4 0

4 0 0 0 0 0

State 4 is the accept state.

State Diagram for "g?ape":



Note: In both diagrams, for simplicity, not all transitions are shown. For states where no explicit transition is shown for a character, the transition leads back to state 0.

3. Time Complexity Analysis

Preprocessing Time Complexity

The preprocessing phase involves building the transition table for the automaton:

1. **Creating the alphabet:** $O(m)$, where m is the total length of all patterns
 - We iterate through each character of each pattern once
2. **Building the automaton:** $O(m \times |\Sigma|)$, where $|\Sigma|$ is the size of the alphabet
 - For each state (at most m states per pattern) and each character in the alphabet, we compute a transition
 - In the worst case, computing a transition may require $O(m)$ time to find the longest proper suffix that is also a prefix

Therefore, the overall preprocessing time complexity is $O(m^2 \times |\Sigma|)$.

For practical implementations, optimizations can reduce this to $O(m \times |\Sigma|)$ using techniques similar to those in the KMP algorithm for computing the failure function.

Searching Time Complexity

The searching phase is more straightforward:

1. For each character in the text, we perform exactly one state transition
2. Each transition takes $O(1)$ time using a hashtable-based implementation
3. When we reach an accept state, verification takes $O(m')$ time, where m' is the length of the matched pattern

Since verification happens relatively infrequently (only when we reach an accept state), the dominant factor is the linear scan through the text.

Therefore, the overall searching time complexity is $O(n)$, where n is the length of the text.

This linear time complexity for the searching phase is one of the major advantages of the finite automata approach, especially for multiple pattern matching, as the time complexity remains linear in the text length regardless of the number or complexity of patterns.

4. Code Explanation

Implementation Logic

My implementation follows these key steps:

1. **Pattern Processing:**
 - Parse input to extract patterns and text lines
 - Build a combined automaton for all patterns
2. **Automaton Construction:**
 - Create a character set (alphabet) from all characters in the patterns
 - For each pattern, build states and transitions
 - Handle wildcards by allowing transitions for any character at wildcard positions
 - Track accept states and associate them with specific patterns
3. **Text Processing:**
 - Process each text line character by character
 - Update the current state based on the transitions
 - When reaching an accept state, verify the match considering wildcards
 - Track all valid matches with their positions
4. **Result Formatting:**
 - Sort matches by position
 - Format output according to the specified requirements

Key Implementation Details

1. **Combined Automaton Structure:** Each pattern gets its own set of states in a combined automaton. This allows efficient searching for multiple patterns in a single pass through the text.
2. **Wildcard Handling:**
 - In transition construction, a wildcard allows transitions for any character
 - During matching, we verify that wildcards correctly substitute for single characters
3. **Case-Insensitive Matching:** The implementation supports case-insensitive matching by normalizing character comparisons.
4. **State Management:** The states are managed using efficient data structures (maps/dictionaries), allowing $O(1)$ lookup time for transitions.
5. **Reset Mechanism:** When encountering characters not in the alphabet or when no valid transition exists, the automaton resets to the initial state, ensuring the algorithm doesn't miss potential matches.

This implementation efficiently combines the classic finite automaton approach with wildcard support, maintaining linear-time text processing while adding the flexibility to match patterns with wildcards.

Q.3.

Part (a): Top k gardens algorithm

To find the registration numbers of the k highest-scoring gardens in $O(|A| + k \cdot \log|A|)$ time:

1. Build a max heap from the unsorted list A in $O(|A|)$ time.
2. Extract the maximum k times, each taking $O(\log|A|)$ time.
3. Return the registration numbers from these k extracted pairs.

Algorithm outline:

1. Convert the unsorted list A into a max heap, with scores as the keys for ordering. This takes $O(|A|)$ time using the heapify operation.
2. Create an empty result array to store registration numbers.
3. For $i = 1$ to k :
 - Extract the maximum element (garden with highest score) from the heap. This takes $O(\log|A|)$ time.
 - Add the registration number (r_i) from this pair to the result array.
4. Return the result array containing the registration numbers of the k highest-scoring gardens.

The total time complexity is $O(|A|)$ for building the max heap plus $O(k \cdot \log|A|)$ for k extract-max operations, resulting in $O(|A| + k \cdot \log|A|)$ as required.

Part (b): Gardens exceeding threshold x

For this part, I need to write pseudocode for a method that runs in $O(n \log n)$ time to extract gardens with scores exceeding a threshold x from a max heap

EXTRACT_GARDENS_ABOVE_THRESHOLD(A, x)

```
    result = [] // To store registration numbers of qualifying gardens
```

```
    if A.heap_size == 0 or A[1].score <= x // Check if heap is empty or max score doesn't exceed
    x
```

```
        return result
```

```
    // Use a queue for breadth-first traversal
```

```
    Q = new Queue()
```

```
    Q.enqueue(1) // Start with root node (index 1)
```

```
    while not Q.isEmpty()
```

```
        i = Q.dequeue()
```

```
        if A[i].score > x
```

```
            result.append(A[i].registration_number)
```

```
        // Add left child to queue if it exists
```

```
        left = 2*i
```

```
        if left <= A.heap_size
```

```
            Q.enqueue(left)
```

```
// Add right child to queue if it exists

right = 2*i + 1

if right <= A.heap_size

    Q.enqueue(right)

return result
```

This approach uses a breadth-first traversal of the max heap, only exploring children of nodes with scores $> x$. Since we're only visiting nodes with scores $> x$ and their immediate children, and each node is processed in constant time, the time complexity is $O(n_x)$ where n_x is the number of gardens with scores exceeding x .

Note: This algorithm assumes the heap property that the parent's score is always greater than or equal to its children's scores. Therefore, once we encounter a node with score $\leq x$, we don't need to explore its children.